

2024年程序设计与思维实践期末考试（回忆版）参考答案

整理：24学术讨论群（syhanjin 老汉 离谱 潜伏 混子 浮萍 東牆 天赐 卡基米 黄鹂 Yasumi Speculator Schwarz Fun10165 Jaaack）以及 24小登对个答案 的群友们

一、单项选择题

题号	1	2	3	4	5	6	7	8	9	10
答案	B	D	C	C	A	A	C	D	D	D

二、填空题

1. (1) `sum < k` (2) `sum == k` (3) `j - 2`
2. (1) `p->next = head` (2) `head = p` (3) `p = head`
3. (1) `row` (2) `a[row][column]`

三、简答题

1. (1) 递归函数是指在函数定义中使用函数自身的方法。
- (2) 递归函数的两个基本要素是递归调用和递归结束条件。递归调用是指在函数内部调用自身，递归结束条件是指确定递归何时结束的条件。
- (3) 优点：能够简化问题的表达，使得代码更加简洁易懂，有时候适合解决复杂问题，能够直观地描述某些问题的解决方案，提高了代码的可读性。
- 缺点：递归调用会增加内存消耗，可能导致性能问题，特别是在递归深度较大时，可能会出现堆栈溢出的情况。

2.

变量类型	调用范围	存储位置	生存时间
全局变量	整个程序	静态存储区	从程序开始运行到结束
静态局部变量	函数内部	静态存储区	从程序开始运行到结束
自动变量	函数内部	动态存储区 (栈)	进入语句块时自动申请内存， 退出时自动释放内存

3. (1)

排序算法	最好	最坏	平均
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$

排序算法	最好	最坏	平均
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$

(2)

排序算法	最坏情况
选择排序	5, 4, 3, 2, 1
快速排序	1, 2, 3, 4, 5

(3)

排序算法	稳定性	反例
选择排序	不稳定	2, 2', 1
快速排序	不稳定	2, 2', 1

4. (1)

数据结构	优点	缺点
单向链表	插入和删除操作效率高，动态内存管理	查找元素效率低，内存消耗较大
数组	查找元素效率高，内存占用小	插入和删除操作效率较低，固定大小

(2)

数据结构	可行性	时间复杂度
单向链表	不能	/
双向链表	能	$O(1)$
循环链表	能	$O(n)$

(3) 参考程序

```
int construct(struct link *lnk)
{
    lnk->head = (struct node*)malloc(sizeof(struct node));
    if (lnk->head == NULL)
    {
        return -1;
    }
    lnk->head->next = lnk->head;
    return 0;
}
```

四、程序设计

1. 参考程序

```

void judge(char T[], int n)
{
    stack *st = (stack*)malloc(sizeof(stack));
    init(st);
    for (int i = 0; i < n; i++)
    {
        char c = T[i];
        if (empty(st))
        {
            push(st, c);
            continue;
        }
        char t;
        pop(st, &t);
        if (t != c)
        {
            push(st, t);
            push(st, c);
        }
    }
    if (empty(st))
    {
        printf("You win!\n");
    }
    else
    {
        printf("You Lose!\n");
    }
}

```

$$2. (1) f(x, n) = \begin{cases} x, & n = 1, \\ f(x, n-1) + (-1)^{n-1}x^n, & n \geq 2 \end{cases} (n \in \mathbb{N}_+)$$

(2) 参考程序

```

double fn(double x, int n)
{
    if (n == 1)
    {
        return x;
    }
    // t = x^n
    double t = x;
    for (int i = 1; i < n; i++)
    {
        t *= x;
    }
    if (n % 2 == 0)
    {
        return fn(x, n - 1) - t;
    }
    else
    {
        return fn(x, n - 1) + t;
    }
}

```

(3) 参考程序

```

#include <stdio.h>

double fn(double x, int n);
// fn定义见(2), 此处略

int main()
{
    double x;
    int n;
    scanf("%lf%d", &x, &n);
    double res;
    res = fn(x, n);
    printf("%.2f", res);
    return 0;
}

```

另解

$$(1) f(x, n) = \begin{cases} x, n = 1, \\ -xf(x, n - 1) + x, n \geq 2 \end{cases} (n \in \mathbb{N}_+)$$

(2) 参考程序

```
double fn(double x, int n)
{
    if (n == 1)
    {
        return x;
    }
    return -x * f(x, n - 1) + x;
}
```

(3) 同上